

700106 / 700120 Lab Book

Vulkan Exercises

Exercise 1: PREPARING FOR TANGENT SPACE

Solution & Reflection: I began by modifying the C++ Vertex struct to include glm::vec3 tangent and glm::vec3 binormal attributes, as shown in the first code screenshot. Following this, I updated the VkVertexInputAttributeDescription entries to account for these two new vec3 attributes, adjusting the binding locations and offsets so Vulkan can correctly read the new data structure from the vertex buffer. Finally, I updated the hard-coded Quad_vertices data (second screenshot) to include specific tangent ({1.0f, 0.0f, 0.0f}) and binormal ({0.0f, 1.0f, 0.0f}) vectors for each vertex, as suggested by the lab for a simple quad.

Reflection: This task was a key foundational step. It made it clear how the TangentBinormal-Normal (TBN) coordinate system is constructed from per-vertex data. Manually adding these vectors for a simple quad was easy, but it highlighted the complexity of generating them for arbitrary meshes. I now understand that these vectors are essential for creating a local coordinate system on the model's surface, which is necessary to correctly orient the normal map

Question: My screenshots show a GenerateTangents function that automatically calculates tangents and binormals based on vertex positions and UV deltas. How does this automatic, code-based calculation compare to using TBN vectors that are pre-calculated and exported directly from a 3D modeling program? Is one method generally preferred for accuracy, especially on models with complex UV seams?

```
// --- Vertex Data ---

struct Vertex {
    glm::vec3 pos;
    glm::vec3 color;
    glm::vec3 normal;
    glm::vec2 texCoord;
    glm::vec3 tangent; // New
    glm::vec3 binormal;
};

134
135 const std::vector<Vertex> Quad_vertices = {
136     {{-0.5f, -0.5f, 0.0f}, {1.0f, 0.0f, 0.0f}, {0.0f, 0.0f, 1.0f}, {0.0f, 1.0f}, {1.0f, 0.0f, 0.0f}, {0.0f, 1.0f, 0.0f}},
137     {{0.5f, -0.5f, 0.0f}, {0.0f, 1.0f, 0.0f}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f, 0.0f}, {0.0f, 1.0f, 0.0f}},
138     {{0.5f, 0.5f, 0.0f}, {0.0f, 0.0f, 1.0f}, {0.0f, 0.0f, 1.0f}, {0.0f, 1.0f}, {1.0f, 0.0f, 0.0f}, {0.0f, 1.0f, 0.0f}},
139     {{-0.5f, 0.5f, 0.0f}, {1.0f, 1.0f, 1.0f}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f, 0.0f}, {0.0f, 1.0f, 0.0f}}
140 };
141
```

Exercise 2: NORMAL MAPPING IMPLEMENTATION

Solution: First, I implemented a `GenerateTangents` C++ function (visible in the screenshots) to automatically calculate the tangent and binormal vectors for the model's vertices, rather than hard-coding them. In the vertex shader (`shader.vert`), I added new in variables for `inTangent` and `inBinormal`. I used these, along with the `inNormal`, to build the TBN matrix. I then used this matrix to transform the world-space light position, view position, and fragment position into tangent space, passing them as out variables (`fragLightPos_tangent`, `fragViewPos_tangent`, etc.) to the fragment shader.

In the fragment shader (`shader.frag`), I added a new uniform `sampler2D normalSampler` at `binding=2`. I sampled this normal map and "unpacked" the resulting vector from its `[0,1]` texture-color range to the `[-1,1]` normalized vector range using the `normalize(N * 2.0 - 1.0)` formula. Finally, I performed all lighting calculations (ambient, diffuse, specular) entirely in tangent space, using this new sampled normal `N` and the interpolated tangent-space vectors. The final screenshot shows the rendered cube with a convincing, bumpy stone appearance.

Reflection: This exercise was a major step. It was insightful to see the lighting calculations move entirely into tangent space. It finally "clicked" *why* tangent space is so useful: the normal map always stores normals relative to a "flat" surface (where Z is "up"), and by transforming the light and view vectors into this *same space*, the lighting logic becomes universal and decoupled from the model's world-space orientation.

Question: In my vertex shader, I transform the light and view *positions* to tangent space and pass them to the fragment shader, where the `L` and `V` *vectors* are calculated after interpolation. Is this approach (interpolating positions) more or less accurate/performant than calculating the `L` and `V` vectors in world-space *first* (in the vertex shader) and then transforming those vectors into tangent space before passing them?

shader.vert

shader.frag

Lab_Tutorial_Template.cpp

```
1      #version 450
2
3      layout(set=0, binding=1) uniform sampler2D colSampler;
4      layout(set=0, binding=2) uniform sampler2D normalSampler;
5
6      layout(push_constant) uniform ObjectData {
7          mat4 model;
8          vec4 ambientMat;
9          vec4 specularMat;
10         float shininess;
11         int texIndex;
12     } pc;
13
14     layout(location=3) in vec2 fragTexCoord;
15     layout(location=4) in vec3 fragLightPos_tangent;
16     layout(location=5) in vec3 fragViewPos_tangent;
17     layout(location=6) in vec3 fragPos_tangent;
18
19     layout(location=0) out vec4 outColor;
20
21     void main() {
22         vec3 albedo = texture(colSampler, fragTexCoord).rgb;
23
24         // Fetch and unpack normal
25         vec3 N = texture(normalSampler, fragTexCoord).rgb;
26         N = normalize(N * 2.0 - 1.0);
27
28         // Convert positions to vectors in tangent space
29         vec3 L = normalize(fragLightPos_tangent - fragPos_tangent);
30         vec3 V = normalize(fragViewPos_tangent - fragPos_tangent);
31         vec3 H = normalize(L + V);
32
33         float diff = max(dot(N, L), 0.0);
34         float spec = 0.0;
35         if (diff > 0.0) {
36             spec = pow(max(dot(N, H), 0.0), pc.shininess);
37         }
38
39         vec3 ambient = pc.ambientMat.rgb * albedo;
40         vec3 diffuse = diff * albedo;
41         vec3 specular = spec * pc.specularMat.rgb;
42
43         vec3 result = ambient + diffuse + specular;
44         outColor = vec4(result, 1.0);
45     }
```

```
shader.vert  # X  shader.frag  Lab_Tutorial_Template.cpp
1      #version 450
2
3      layout(set=0,binding=0) uniform SceneUBO {
4          mat4 view;
5          mat4 proj;
6          vec3 lightPos1;
7          vec3 lightPos2;
8          vec3 eyePos;
9          vec2 texSize;
10         int  filterMode;
11     } ubo;
12
13     layout(push_constant) uniform ObjectData {
14         mat4 model;
15         vec4 ambientMat;
16         vec4 specularMat;
17         float shininess;
18         int  texIndex;
19     } pc;
20
21     layout(location=0) in vec3 inPosition;
22     layout(location=1) in vec3 inColor;
23     layout(location=2) in vec3 inNormal;
24     layout(location=3) in vec2 inTexCoord;
25     layout(location=4) in vec3 inTangent;
26     layout(location=5) in vec3 inBinormal;
27
28     layout(location=0) out vec3 passColor;           // (optional if needed)
29     layout(location=1) out vec3 passNormal;          // (debug)
30     layout(location=2) out vec3 passWorldPos;        // (debug)
31     layout(location=3) out vec2 fragTexCoord;
32     layout(location=4) out vec3 fragLightPos_tangent;
33     layout(location=5) out vec3 fragViewPos_tangent;
34     layout(location=6) out vec3 fragPos_tangent;
35
36     void main() {
37         mat4 MVP = ubo.proj * ubo.view * pc.model;
38         gl_Position = MVP * vec4(inPosition, 1.0);
39
40         vec3 worldPos = vec3(pc.model * vec4(inPosition,1.0));
41         passWorldPos = worldPos;
42         fragTexCoord = inTexCoord;
43
44         // Normal matrix
45         mat3 normalMatrix = transpose(inverse(mat3(pc.model)));
46         vec3 T = normalize(normalMatrix * inTangent);
47         vec3 B = normalize(normalMatrix * inBinormal);
48         vec3 N = normalize(normalMatrix * inNormal);
49     }
```

```

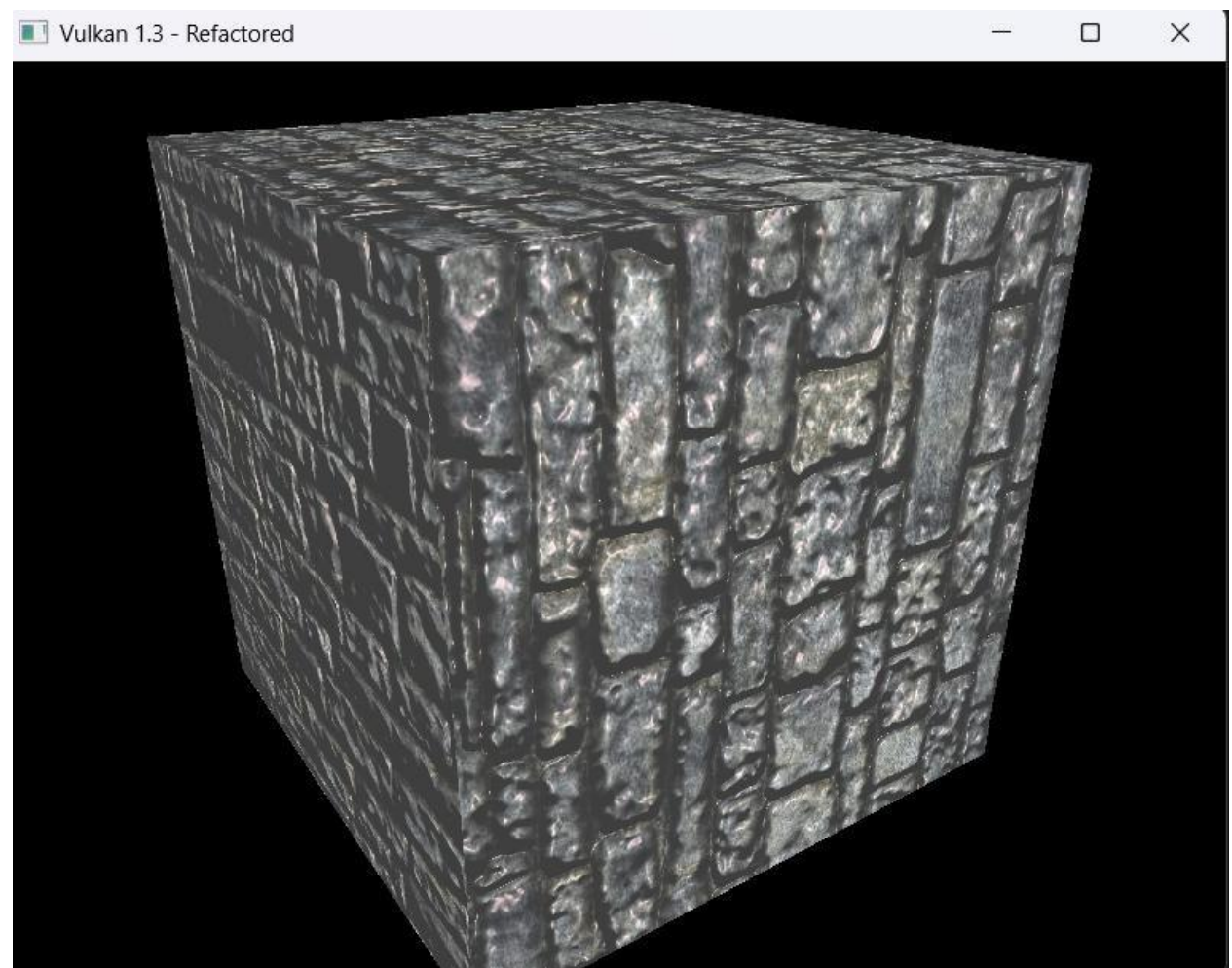
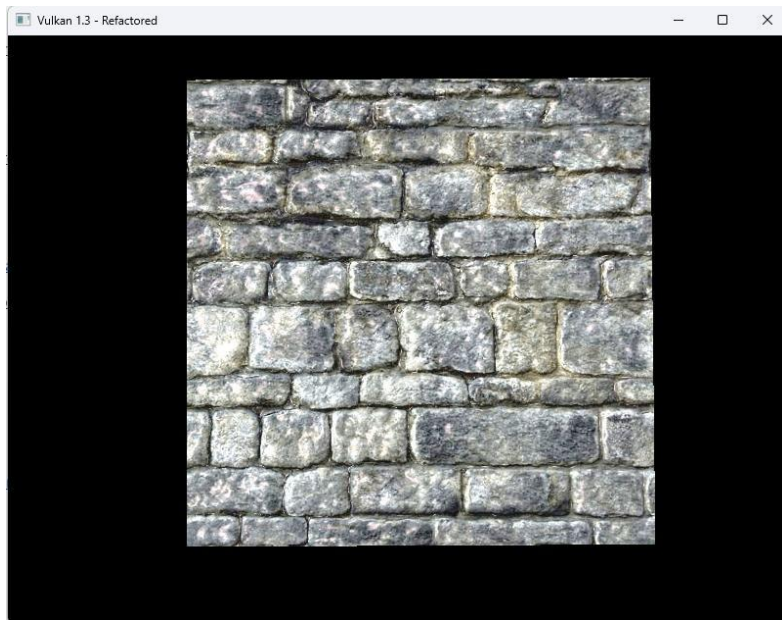
49
50         mat3 TBN = transpose(mat3(T,B,N)); // world->tangent
51
52         // Positions (can also send vectors subtracting fragPos)
53         vec3 lightPos_world = ubo.lightPos1;
54         vec3 viewPos_world  = ubo.eyePos;
55
56         fragLightPos_tangent = TBN * lightPos_world;
57         fragViewPos_tangent  = TBN * viewPos_world;
58         fragPos_tangent      = TBN * worldPos;
59
60         passColor  = inColor;
61         passNormal = N;

```

```

239 static void GenerateTangents(std::vector<Vertex>& verts, const std::vector<uint16_t>& idx) {
240     // zero
241     for (auto& v : verts) {
242         v.tangent = glm::vec3(0);
243         v.binormal = glm::vec3(0);
244     }
245     for (size_t i = 0; i < idx.size(); i += 3) {
246         Vertex& v0 = verts[idx[i + 0]];
247         Vertex& v1 = verts[idx[i + 1]];
248         Vertex& v2 = verts[idx[i + 2]];
249
250         glm::vec3 p0 = v0.pos;
251         glm::vec3 p1 = v1.pos;
252         glm::vec3 p2 = v2.pos;
253
254         glm::vec2 uv0 = v0.texCoord;
255         glm::vec2 uv1 = v1.texCoord;
256         glm::vec2 uv2 = v2.texCoord;
257
258         glm::vec3 dp1 = p1 - p0;
259         glm::vec3 dp2 = p2 - p0;
260         glm::vec2 duv1 = uv1 - uv0;
261         glm::vec2 duv2 = uv2 - uv0;
262
263         float r = (duv1.x * duv2.y - duv1.y * duv2.x);
264         if (fabs(r) < 1e-8f) r = 1.0f; else r = 1.0f / r;
265
266         glm::vec3 T = (dp1 * duv2.y - dp2 * duv1.y) * r;
267         glm::vec3 B = (dp2 * duv1.x - dp1 * duv2.x) * r;
268
269         v0.tangent += T; v1.tangent += T; v2.tangent += T;
270         v0.binormal += B; v1.binormal += B; v2.binormal += B;
271     }
272     for (auto& v : verts) {
273         v.tangent = glm::normalize(v.tangent);
274         v.binormal = glm::normalize(v.binormal);
275         // (Optional) re-orthogonalize: v.tangent = normalize(v.tangent - v.normal * dot(v.normal,v.tangent));
276     }
277 }

```

```

// These functions also use the command pool
createTextureFromFile("stones.png");
//createSecondTextureFromFile("rock.jpg");
//createNormalTextureFromFile("rockNormal.png");
createHeightTextureFromFile("rockheight.png");

createVertexBuffer():

```

```

VkDescriptorImageInfo colorInfo{};
colorInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
colorInfo.imageView = textureImageView;
colorInfo.sampler = textureSampler;

```

```

1943
1944 void HelloTriangleApplication::createNormalTextureFromFile(const std::string& filename) {
1945     int texWidth, texHeight, texChannels;
1946     stbi_uc* pixels = stbi_load(filename.c_str(), &texWidth, &texHeight, &texChannels, STBI_rgb_alpha);
1947     if (!pixels) throw std::runtime_error("failed to load normal map: " + filename);
1948
1949     VkDeviceSize imageSize = static_cast<VkDeviceSize>(texWidth) * texHeight * 4;
1950
1951     VkBuffer stagingBuffer;
1952     VkDeviceMemory stagingMemory;
1953     createBuffer(imageSize,
1954         VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
1955         VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
1956         stagingBuffer, stagingMemory);
1957
1958     void* data;
1959     vkMapMemory(device, stagingMemory, 0, imageSize, 0, &data);
1960     memcpy(data, pixels, (size_t)imageSize);
1961     vkUnmapMemory(device, stagingMemory);
1962     stbi_image_free(pixels);
1963
1964     VkImageCreateInfo imgInfo{};
1965     imgInfo.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
1966     imgInfo.imageType = VK_IMAGE_TYPE_2D;
1967     imgInfo.extent = { (uint32_t)texWidth, (uint32_t)texHeight, 1 };
1968     imgInfo.mipLevels = 1;
1969     imgInfo.arrayLayers = 1;
1970     imgInfo.format = VK_FORMAT_R8G8B8A8_UNORM; // normal maps: UNORM (no SRGB)
1971     imgInfo.tiling = VK_IMAGE_TILING_OPTIMAL;
1972     imgInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
1973     imgInfo.usage = VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT;
1974     imgInfo.samples = VK_SAMPLE_COUNT_1_BIT;
1975     imgInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
1976
1977     if (vkCreateImage(device, &imgInfo, nullptr, &normalTextureImage) != VK_SUCCESS)
1978         throw std::runtime_error("failed to create normal image");
1979
1980     VkMemoryRequirements memReq;
1981     vkGetImageMemoryRequirements(device, normalTextureImage, &memReq);
1982
1983     VkMemoryAllocateInfo alloc{};
1984     alloc.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
1985     alloc.allocationSize = memReq.size;
1986     alloc.memoryTypeIndex = findMemoryType(memReq.memoryTypeBits, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
1987     if (vkAllocateMemory(device, &alloc, nullptr, &normalTextureImageMemory) != VK_SUCCESS)
1988         throw std::runtime_error("failed to alloc normal image memory");

```

Exercise 3: HEIGHT MAP BUMPY EFFECT

Question: Improve visual quality by moving the lighting logic to the fragment shader, allowing for smoother, more accurate calculations.

Solution: I implemented an alternative bump effect that generates normals on-the-fly from a grayscale height map instead of using a pre-calculated normal map.

In the C++ code, I added the logic to load a new texture ("rockheight.png") and create the corresponding `VkImage`, `VkImageView`, and `VkSampler` for it, as seen in the `createHeightTextureFromFile` screenshots. I then updated the descriptor set layout and descriptor pool to account for this new `heightSampler`, binding it to slot 2 (as seen in the `shader.frag` merge conflict screenshot).

The main work was in the **fragment shader** (`shader.frag`). I sampled the new `heightSampler` to get a single grayscale value, `h`. Using this height, I calculated its screenspace derivatives with `float dHx = dFdx(h);` and `float dHy = dFdy(h);`. These derivatives represent the slope of the height map at that fragment. I used these slopes to construct a new normal vector, which was then used in the world-space Phong lighting equations to create the final bumpy look. The final screenshot shows the "wavy" distortion effect this technique produces.

Reflection: This was a fascinating alternative to normal mapping. It's powerful that you can derive 3D surface information from a simple 2D grayscale texture. Using the `dFdx` and `dFdy` functions was new to me, and it was interesting to see how these fragment-shader-only functions can be used to calculate geometric properties. The visual result is noticeably different from normal mapping—it looks more like a true displacement or distortion, as seen in the output. This exercise helped me understand the practical difference between *storing* explicit normals (normal map) and *deriving* them from slopes (height map).

```
232
233     VkImage heightTextureImage = VK_NULL_HANDLE;
234     VkDeviceMemory heightTextureImageMemory = VK_NULL_HANDLE;
235     VkImageView heightTextureImageView = VK_NULL_HANDLE;
236     VkSampler heightTextureSampler = VK_NULL_HANDLE;
237
```



```

2011
2012 void HelloTriangleApplication::createHeightTextureFromFile(const std::string& filename)
2013 {
2014     int w, h, c;
2015     stbi_uc* pixels = stbi_load(filename.c_str(), &w, &h, &c, STBI_rgb_alpha);
2016     if (!pixels) throw std::runtime_error("failed to load height map: " + filename);
2017     VkDeviceSize size = (VkDeviceSize)w * h * 4;
2018
2019     VkBuffer staging;
2020     VkDeviceMemory stagingMem;
2021     createBuffer(size, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
2022         VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
2023         staging, stagingMem);
2024
2025     void* data;
2026     vkMapMemory(device, stagingMem, 0, size, 0, &data);
2027     memcpy(data, pixels, (size_t)size);
2028     vkUnmapMemory(device, stagingMem);
2029     stbi_image_free(pixels);
2030
2031     VkImageCreateInfo img{};
2032     img.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
2033     img.imageType = VK_IMAGE_TYPE_2D;
2034     img.extent = { (uint32_t)w, (uint32_t)h, 1 };
2035     img.mipLevels = 1;
2036     img.arrayLayers = 1;
2037     img.format = VK_FORMAT_R8G8B8A8_UNORM; // height map UNORM
2038     img.tiling = VK_IMAGE_TILING_OPTIMAL;
2039     img.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
2040     img.usage = VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT;
2041     img.samples = VK_SAMPLE_COUNT_1_BIT;
2042     img.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
2043
2044     if (vkCreateImage(device, &img, nullptr, &heightTextureImage) != VK_SUCCESS)
2045         throw std::runtime_error("failed to create height image");
2046     VkMemoryRequirements req;
2047     vkGetImageMemoryRequirements(device, heightTextureImage, &req);
2048     VkMemoryAllocateInfo alloc{};
2049     alloc.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
2050     alloc.allocationSize = req.size;
2051     alloc.memoryTypeIndex = findMemoryType(req.memoryTypeBits, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
2052     if (vkAllocateMemory(device, &alloc, nullptr, &heightTextureImageMemory) != VK_SUCCESS)
2053         throw std::runtime_error("failed to alloc height image memory");
2054     vkBindImageMemory(device, heightTextureImage, heightTextureImageMemory, 0);
2055
2056     transitionImageLayout(heightTextureImage,
2057         VK_IMAGE_LAYOUT_UNDEFINED,
2058         VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
2059         VK_IMAGE_ASPECT_COLOR_BIT);
2060
2061     copyBufferToImage(staging, heightTextureImage, (uint32_t)w, (uint32_t)h);
2062
2063     transitionImageLayout(heightTextureImage,
2064         VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
2065         VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL,
2066         VK_IMAGE_ASPECT_COLOR_BIT);
2067
2068     vkDestroyBuffer(device, staging, nullptr);
2069     vkFreeMemory(device, stagingMem, nullptr);
2070
2071     heightTextureImageView = createImageView(heightTextureImage, VK_FORMAT_R8G8B8A8_UNORM, VK_IMAGE_ASPECT_COLOR_BIT);
2072     createTextureSampler(heightTextureSampler, textureFilterMode);
2073 }

```

```
2235     VkDescriptorImageInfo heightInfo{};
2236     heightInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
2237     heightInfo.imageView = heightTextureImageView;
2238     heightInfo.sampler = heightTextureSampler;
2239
2240     std::array<VkWriteDescriptorSet, 2> writes{};
2241
2242     // Rebind coin sampler
2243     writes[0].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
2244     writes[0].dstSet = descriptorSets[i];
2245     writes[0].dstBinding = 1;
2246     writes[0].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
2247     writes[0].descriptorCount = 1;
2248     writes[0].pImageInfo = &colorInfo;
2249
2250     // Rebind tile sampler
2251     writes[1].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
2252     writes[1].dstSet = descriptorSets[i];
2253     writes[1].dstBinding = 2;
2254     writes[1].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
2255     writes[1].descriptorCount = 1;
2256     writes[1].pImageInfo = &heightInfo;
```

shader.vert × shader.frag × Lab_Tutorial_Template.cpp

File contains merge conflicts. Open Merge Editor

```
1 #version 450
2 #define DEBUG_VIEW 1 // 0=shaded, 1=height grayscale, 2=perturbed normal RGB
3
4 layout(set=0,binding=0) uniform SceneUBO {
5     mat4 view;
6     mat4 proj;
7     vec3 lightPos1;
8     vec3 lightPos2;
9     vec3 eyePos;
10    vec2 texSize;
11    int filterMode;
12 } ubo;
13 <<<<<< Updated upstream
14
15 layout(set=0,binding=1) uniform sampler2D colSampler; // diffuse
16 layout(set=0,binding=2) uniform sampler2D heightSampler; // height map
17 =====
18 >>>>>> Stashed changes
19
20 layout(push_constant) uniform ObjectData {
21     mat4 model;
22     vec4 ambientMat;
23     vec4 specularMat;
24     float shininess;
25     int texIndex;
26 } pc;
27
28 <<<<<< Updated upstream
29 layout(location=0) in vec3 fragWorldPos;
30 layout(location=1) in vec3 fragWorldNormal;
31 layout(location=2) in vec2 fragTexCoord;
32
33 layout(location=0) out vec4 outColor;
34
35 const float bumpHeight = 0.4f; // between 0.1 and 1.0
36
37 void main() {
38     vec3 albedo = texture(colSampler, fragTexCoord).rgb;
39
40     // Height (grayscale)
41     float h = texture(heightSampler, fragTexCoord).r;
42
43     // Debug view: height map as grayscale
44     #if DEBUG_VIEW == 1
45         outColor = vec4(vec3(h), 1.0);
46         return;
47     #endif
```

File contains merge conflicts. Open Merge Editor

```

49 // Derivatives in screen space
50 float dHx = dFdx(h);
51 float dHy = dFdy(h);
52
53 // Pseudo "screen" normal, then remap onto a basis around the world normal
54 vec3 screenNormal = normalize(vec3(-dHx, -dHy, bumpHeight));
55
56 vec3 N = normalize(fragWorldNormal);
57 vec3 helper = abs(N.z) < 0.9 ? vec3(0,0,1) : vec3(0,1,0);
58 vec3 T = normalize(cross(helper, N));
59 vec3 B = normalize(cross(N, T));
60
61 vec3 perturbed = normalize(screenNormal.x * T + screenNormal.y * B + screenNormal.z * N);
62
63 // Debug view: show perturbed normal in RGB
64 #if DEBUG_VIEW == 2
65     outColor = vec4(perturbed * 0.5 + 0.5, 1.0);
66     return;
67 #endif
68
69 // Shaded result (world-space Phong)
70 vec3 lightPos = ubo.lightPos1;
71 vec3 L = normalize(lightPos - fragWorldPos);
72 vec3 V = normalize(ubo.eyePos - fragWorldPos);
73 vec3 H = normalize(L + V);
74
75 float diff = max(dot(perturbed, L), 0.0);
76 float spec = diff > 0.0 ? pow(max(dot(perturbed, H), 0.0), pc.shininess) : 0.0;
77
78 =====
79 layout(set=0, binding=1) uniform sampler2D colSampler;
80 layout(set=0, binding=2) uniform sampler2D normalSampler;
81
82 layout(location=3) in vec2 fragTexCoord;
83 layout(location=4) in vec3 fragLightPos_tangent;
84 layout(location=5) in vec3 fragViewPos_tangent;
85 layout(location=6) in vec3 fragPos_tangent;
86
87 layout(location=0) out vec4 outColor;
88
89 void main()
90 {
91     // Base color
92     vec3 albedo = texture(colSampler, fragTexCoord).rgb;
93
94     // Unpack normal and strengthen bumps (increase to make darker, more pronounced crevices)
95     float normalStrength = 2.0; // try 1.5-3.0
96     vec3 nMap = texture(normalSampler, fragTexCoord).rgb * 2.0 - 1.0;
97     vec3 N = normalize(vec3(nMap.xy * normalStrength, nMap.z));
98
99     // Tangent-space lighting vectors
100     vec3 L = normalize(fragLightPos_tangent - fragPos_tangent);
101     vec3 V = normalize(fragViewPos_tangent - fragPos_tangent);
102     vec3 H = normalize(L + V);
103
104     float NdotL = max(dot(N, L), 0.0);
105     float NdotH = max(dot(N, H), 0.0);
106
107     >>>>>> Stashed changes
108
109     // Darker ambient to match the right image
110     vec3 ambient = pc.ambientMat.rgb * 0.05;
111
112     vec3 diffuse = albedo * NdotL;
113
114     float specPow = max(pc.shininess, 1.0);
115     float spec = (NdotL > 0.0) ? pow(NdotH, specPow) : 0.0;
116     vec3 specular = pc.specularMat.rgb * spec;
117
118     // Subtle self-occlusion from the normal's Z to darken crevices further
119     float occl = mix(0.6, 1.0, clamp(N.z, 0.0, 1.0));
120
121     vec3 color = (ambient + (diffuse + specular) * occl);
122     outColor = vec4(color, 1.0);
123 }

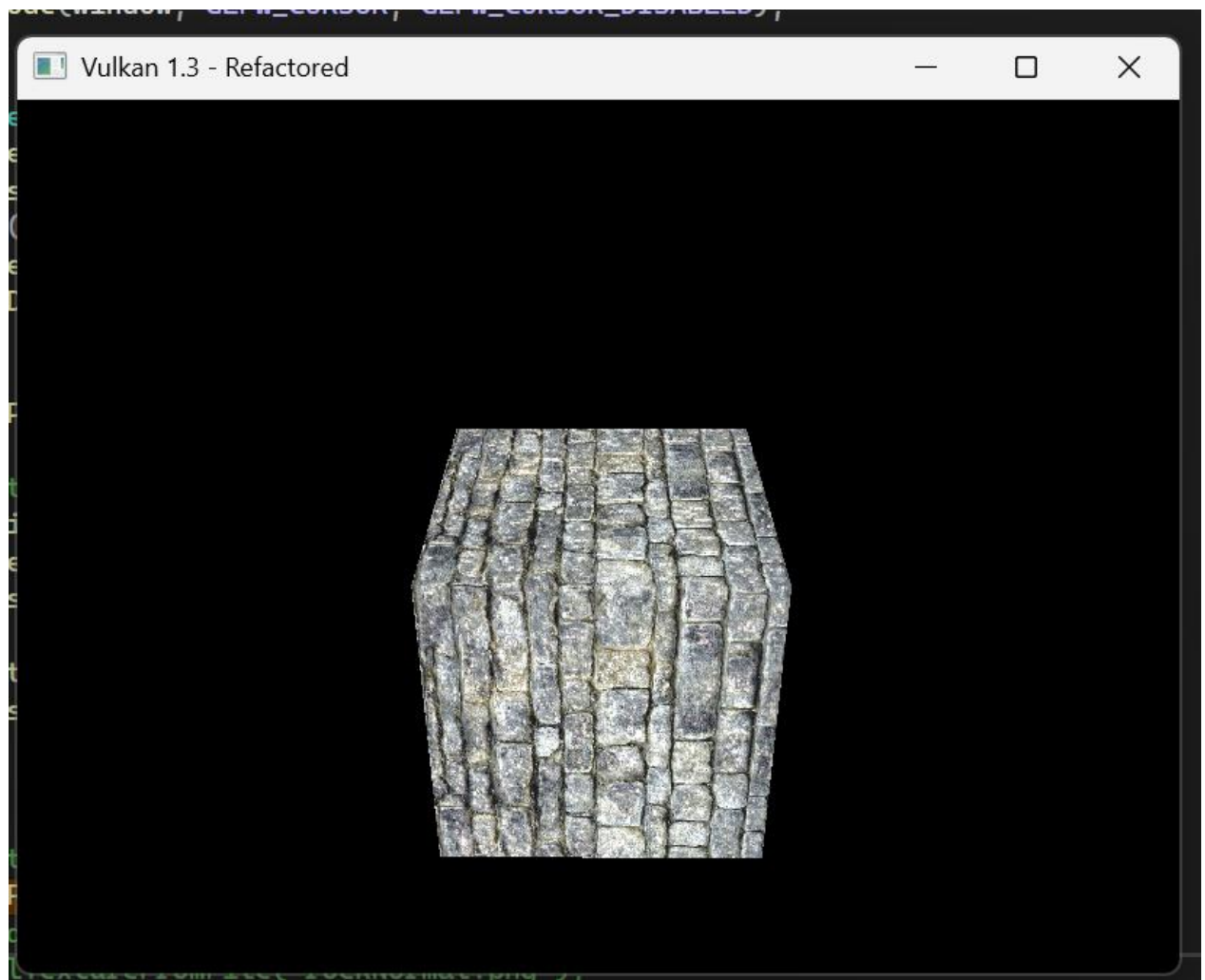
```



```

shader.vert  X  shader.frag  Lab_Tutorial_Template.cpp
1      #version 450
2
3      layout(set=0, binding=0) uniform SceneUBO {
4          mat4 view;
5          mat4 proj;
6          vec3 lightPos1;
7          vec3 lightPos2;
8          vec3 eyePos;
9          vec2 texSize;
10         int  filterMode;
11     } ubo;
12
13     layout(push_constant) uniform ObjectData {
14         mat4 model;
15         vec4 ambientMat;
16         vec4 specularMat;
17         float shininess;
18         int  texIndex;
19     } pc;
20
21     layout(location=0) in vec3 inPosition;
22     layout(location=1) in vec3 inColor;
23     layout(location=2) in vec3 inNormal;
24     layout(location=3) in vec2 inTexCoord;
25
26     layout(location=0) out vec3 fragWorldPos;
27     layout(location=1) out vec3 fragWorldNormal;
28     layout(location=2) out vec2 fragTexCoord;
29
30     void main() {
31         vec4 worldPos = pc.model * vec4(inPosition, 1.0);
32         fragWorldPos = worldPos.xyz;
33
34         mat3 normalMatrix = transpose(inverse(mat3(pc.model)));
35         fragWorldNormal = normalize(normalMatrix * inNormal);
36
37         fragTexCoord = inTexCoord;
38
39         gl_Position = ubo.proj * ubo.view * worldPos;
40     }

```



Exercise 4: PROCEDURAL NORMAL MAPPING

Solution: The solution, visible in the shader.frag screenshot, involved two new helper functions. First, a computeTBN function was implemented, which cleverly builds the TBN matrix on-the-fly using screen-space derivatives ($dFdx$, $dFdy$) of the fragment's world position and texture coordinates. This advanced technique completely removes the need to pass tangent and binormal vectors from the C++ application.

Second, I wrote a HemisphereBumpTS function. This function mathematically generates a tangent-space normal vector (N_tan) based on the fragment's uv coordinate. It uses fract and dot operations to create a repeating grid of hemispheres.

In the main() function, I first called computeTBN to get the TBN matrix. Then, I called HemisphereBumpTS to get the procedural N_tan . I transformed this procedural normal into world space ($vec3\ N = normalize(TBN * N_tan);$) and used this new N for all world-space Phong lighting calculations, which I also updated to support two light sources. The final screenshot shows the cube rendered with this procedural "rivet" pattern, with each bump correctly catching specular highlights from both lights.

Reflection: This was the most complex but also the most rewarding exercise. Calculating the TBN matrix in the fragment shader using derivatives is a powerful optimization that simplifies the vertex data pipeline significantly. Writing the HemisphereBumpTS function was a great look into procedural generation, showing how simple math can create complex, tileable geometric patterns from scratch.

Question: The computeTBN function in my shader (Image 24) uses $dFdx/dFdy$ on fragWorldPos and fragTexCoord. This seems very efficient. Are there any visual quality downsides or artifacts to this on-the-fly TBN generation compared to using pre-calculated per-vertex tangents, especially on models with sharp UV seams or non-uniform UV scaling?

```
shader.vert  shader.frag  X  Lab_Tutorial_Template.cpp
File contains merge conflicts.  Open Merge Editor

1      #version 450
2
3      layout(set=0, binding=0) uniform SceneUBO {
4          mat4 view;
5          mat4 proj;
6          vec3 lightPos1;
7          vec3 lightPos2;
8          vec3 eyePos;
9          vec2 texSize;
10         int  filterMode;
11     } ubo;
12
13     layout(push_constant) uniform ObjectData {
14         mat4 model;
15         vec4 ambientMat;
16         vec4 specularMat;
17         float shininess;
18         int  texIndex;
19     } pc;
20
21     // Optional albedo texture (use a bound texture if you have one)
22     layout(set=0, binding=1) uniform sampler2D colSampler;
23
24     layout(location=0) in vec3 fragWorldPos;
25     layout(location=1) in vec3 fragWorldNormal;
26     layout(location=2) in vec2 fragTexCoord;
27
28     layout(location=0) out vec4 outColor;
29
30     // Build TBN from screen-space derivatives (no vertex tangents needed)
31     mat3 computeTBN(vec3 N, vec3 P, vec2 uv)
32     {
33         vec3 dpdx = dFdx(P);
34         vec3 dpdy = dFdy(P);
35         vec2 dtdx = dFdx(uv);
36         vec2 dtdy = dFdy(uv);
37
38         float r = 1.0 / (dtdx.x * dtdy.y - dtdx.y * dtdy.x);
39         vec3 T = (dpdx * dtdy.y - dpdy * dtdx.y) * r;
40         vec3 B = (dpdy * dtdx.x - dpdx * dtdy.x) * r;
41
42         // Orthonormalize
43         vec3 n = normalize(N);
44         vec3 t = normalize(T - n * dot(n, T));
45         vec3 b = normalize(cross(n, t));
46         return mat3(t, b, n);
47     }
```


File contains merge conflicts. Open Merge Editor

```

49 // Generate a tiled hemisphere normal in tangent space
50 vec3 hemisphereBumpTS(vec2 uv, float tiles, float radius)
51 {
52     // Local uv in [-1, 1] within each tile
53     vec2 uvLocal = fract(uv * tiles) * 2.0 - 1.0;
54
55     float R2 = radius * radius;
56     float r2 = dot(uvLocal, uvLocal);
57
58     // Default: flat surface
59     vec3 N_tan = vec3(0.0, 0.0, 1.0);
60
61     if (r2 < R2)
62     {
63         // Hemisphere of radius=1 mapped onto disk of radius "radius"
64         vec2 p = uvLocal / radius;
65         float z = sqrt(max(0.0, 1.0 - dot(p, p)));
66         N_tan = normalize(vec3(p.x, p.y, z));
67     }
68     return N_tan;
69 }
70
71 void main()
72 {
73     // Base color (fallback to gray if no texture bound)
74     vec3 albedo = texture(colSampler, fragTexCoord).rgb;
75     if (albedo == vec3(0.0)) { albedo = vec3(0.6); }
76
77     // TBN from derivatives
78     mat3 TBN = computeTBN(normalize(fragWorldNormal), fragWorldPos, fragTexCoord);
79
80     // Procedural normal in tangent space
81     float tiles = 5.0; // number of bumps per UV axis
82     float radius = 0.7; // bump radius inside each tile (0..1)
83     vec3 N_tan = hemisphereBumpTS(fragTexCoord, tiles, radius);
84
85     // Transform to world space
86     vec3 N = normalize(TBN * N_tan);
87
88     // Phong lighting with two lights
89     vec3 V = normalize(ubo.eyePos - fragWorldPos);
90
91     vec3 L1 = normalize(ubo.lightPos1 - fragWorldPos);
92     vec3 H1 = normalize(L1 + V);
93     float NdotL1 = max(dot(N, L1), 0.0);
94     float spec1 = (NdotL1 > 0.0) ? pow(max(dot(N, H1), 0.0), max(pc.shininess, 1.0)) : 0.0;
95
96     vec3 L2 = normalize(ubo.lightPos2 - fragWorldPos);
97     vec3 H2 = normalize(L2 + V);
98     float NdotL2 = max(dot(N, L2), 0.0);
99     float spec2 = (NdotL2 > 0.0) ? pow(max(dot(N, H2), 0.0), max(pc.shininess, 1.0)) : 0.0;
100
101     vec3 ambient = pc.ambientMat.rgb * 0.08;
102     vec3 diffuse = albedo * (NdotL1 + NdotL2);
103     vec3 specular = pc.specularMat.rgb * (spec1 + spec2);
104
105     vec3 color = ambient + diffuse + specular;
106     outColor = vec4(color, 1.0);
107 }

```



Exercise 5: TEXTURE WRAPPING MODE

Solution & Reflection: I extended the Vulkan rendering pipeline to support binding multiple textures simultaneously, which is the C++-side preparation needed for advanced techniques like parallax mapping.

To prepare for sampling from color, normal, and height maps in a single shader, I had to update the descriptor set configuration.

Layout: In `createDescriptorSetLayout` (Image 31), I added three separate `VkDescriptorSetLayoutBinding` entries for combined image samplers: `colorBinding` (`binding=1`), `normalBinding` (`binding=2`), and `heightBinding` (`binding=3`).

Pool: In `createDescriptorPool` (Image 28), I updated the `VkDescriptorPoolSize` for `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER` to allocate `MAX_FRAMES_IN_FLIGHT * 3`, ensuring the pool has enough space for all three textures per frame.

Sets: In `createDescriptorSets` (Image 30) and `updateDescriptorSetsSampler` (Image 29), I created and wrote three distinct `VkDescriptorImageInfo` structures (`colorInfo`, `normalInfo`, `heightInfo`) to each descriptor set, binding the correct texture view and sampler to their corresponding slots (1, 2, and 3).

Reflection: This task was a great exercise in scaling up the application's resource management. It solidified my understanding of how descriptor sets act as a "bundle" of resources for a shader pipeline. Adding multiple textures was a straightforward process of carefully updating the layout, pool, and set-writing functions. This setup is far more flexible and is clearly the foundation for any modern material system that might use PBR textures (albedo, normal, metallic, roughness, AO) or, in this case, parallax mapping.

Question: I've now set up the C++ side to bind the color, normal, and height maps needed for parallax mapping. The lab describes this technique as displacing UVs based on the view angle and height map. This sounds computationally expensive. Is parallax mapping (or more advanced versions like steep parallax or relief mapping) still a common technique in AAA games, or have modern engines largely replaced it with geometry-based solutions like tessellation and displacement shaders?

```

1109
1110 void HelloTriangleApplication::createDescriptorPool() {
1111     std::array<VkDescriptorPoolSize, 2> poolSizes{};
1112
1113     // Uniform buffers
1114     poolSizes[0].type = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
1115     poolSizes[0].descriptorCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
1116
1117     // We now have 3 sampled images per set (color + normal + height)
1118     poolSizes[1].type = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
1119     poolSizes[1].descriptorCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT) * 3;
1120
1121     VkDescriptorPoolCreateInfo poolInfo{};
1122     poolInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO;
1123     poolInfo.poolSizeCount = static_cast<uint32_t>(poolSizes.size());
1124     poolInfo.pPoolSizes = poolSizes.data();
1125     poolInfo.maxSets = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
1126
1127     if (vkCreateDescriptorPool(device, &poolInfo, nullptr, &descriptorPool) != VK_SUCCESS) {
1128         throw std::runtime_error("failed to create descriptor pool!");
1129     }
1130 }

```

```

2175
2176 void HelloTriangleApplication::updateDescriptorSetsSampler() {
2177     if (descriptorSets.empty()) return;
2178
2179     for (size_t i = 0; i < descriptorSets.size(); ++i) {
2180         VkDescriptorImageInfo colorInfo{};
2181         colorInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
2182         colorInfo.imageView = textureImageView;
2183         colorInfo.sampler = textureSampler;
2184
2185         VkDescriptorImageInfo normalInfo{};
2186         normalInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
2187         normalInfo.imageView = normalTextureImageView;
2188         normalInfo.sampler = normalTextureSampler;
2189
2190         VkDescriptorImageInfo heightInfo{};
2191         heightInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
2192         heightInfo.imageView = heightTextureImageView;
2193         heightInfo.sampler = heightTextureSampler;
2194
2195         std::array<VkWriteDescriptorSet, 3> writes{};
2196
2197         // Color
2198         writes[0].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
2199         writes[0].dstSet = descriptorSets[i];
2200         writes[0].dstBinding = 1;
2201         writes[0].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
2202         writes[0].descriptorCount = 1;
2203         writes[0].pImageInfo = &colorInfo;
2204
2205         // Normal
2206         writes[1].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
2207         writes[1].dstSet = descriptorSets[i];
2208         writes[1].dstBinding = 2;
2209         writes[1].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
2210         writes[1].descriptorCount = 1;
2211         writes[1].pImageInfo = &normalInfo;
2212
2213         // Height
2214         writes[2].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
2215         writes[2].dstSet = descriptorSets[i];
2216         writes[2].dstBinding = 3;
2217         writes[2].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
2218         writes[2].descriptorCount = 1;
2219         writes[2].pImageInfo = &heightInfo;

```



```

1131
1132 void HelloTriangleApplication::createDescriptorSets() {
1133     std::vector<VkDescriptorSetLayout> layouts(MAX_FRAMES_IN_FLIGHT, descriptorSetLayout);
1134
1135     VkDescriptorSetAllocateInfo allocInfo{};
1136     allocInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO;
1137     allocInfo.descriptorPool = descriptorPool;
1138     allocInfo.descriptorSetCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
1139     allocInfo.pSetLayouts = layouts.data();
1140
1141     descriptorSets.resize(MAX_FRAMES_IN_FLIGHT);
1142     if (vkAllocateDescriptorSets(device, &allocInfo, descriptorSets.data()) != VK_SUCCESS) {
1143         throw std::runtime_error("Failed to allocate descriptor sets!");
1144     }
1145
1146     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
1147         VkDescriptorBufferInfo bufferInfo{};
1148         bufferInfo.buffer = uniformBuffers[i];
1149         bufferInfo.offset = 0;
1150         bufferInfo.range = sizeof(SceneUniformBufferObject);
1151
1152         VkDescriptorImageInfo colorInfo{};
1153         colorInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
1154         colorInfo.imageView = textureImageView;
1155         colorInfo.sampler = textureSampler;
1156
1157         VkDescriptorImageInfo normalInfo{};
1158         normalInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
1159         normalInfo.imageView = normalTextureImageView;
1160         normalInfo.sampler = normalTextureSampler;
1161
1162         VkDescriptorImageInfo heightInfo{};
1163         heightInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
1164         heightInfo.imageView = heightTextureImageView;
1165         heightInfo.sampler = heightTextureSampler;
1166
1167         std::array<VkWriteDescriptorSet, 4> writes{};
1168
1169         // UBO
1170         writes[0].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
1171         writes[0].dstSet = descriptorSets[i];
1172         writes[0].dstBinding = 0;
1173         writes[0].descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
1174         writes[0].descriptorCount = 1;
1175         writes[0].pBufferInfo = &bufferInfo;
1176
1177

```

```

873
874 void HelloTriangleApplication::createDescriptorSetLayout() {
875     VkDescriptorSetLayoutBinding uboLayoutBinding{};
876     uboLayoutBinding.binding = 0;
877     uboLayoutBinding.descriptorCount = 1;
878     uboLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
879     uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT;
880
881     // Binding 1: color/albedo texture
882     VkDescriptorSetLayoutBinding colorBinding{};
883     colorBinding.binding = 1;
884     colorBinding.descriptorCount = 1;
885     colorBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
886     colorBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;
887
888     // Binding 2: normal map
889     VkDescriptorSetLayoutBinding normalBinding{};
890     normalBinding.binding = 2;
891     normalBinding.descriptorCount = 1;
892     normalBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
893     normalBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;
894
895     // Binding 3: height map
896     VkDescriptorSetLayoutBinding heightBinding{};
897     heightBinding.binding = 3;
898     heightBinding.descriptorCount = 1;
899     heightBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
900     heightBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;
901
902     std::array<VkDescriptorSetLayoutBinding, 4> bindings = {
903         uboLayoutBinding, colorBinding, normalBinding, heightBinding
904     };
905
906     VkDescriptorSetLayoutCreateInfo layoutInfo{};
907     layoutInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
908     layoutInfo.bindingCount = static_cast<uint32_t>(bindings.size());
909     layoutInfo.pBindings = bindings.data();
910
911     if (vkCreateDescriptorSetLayout(device, &layoutInfo, nullptr, &descriptorSetLayout) != VK_SUCCESS) {
912         throw std::runtime_error("Failed to create descriptor set layout!");
913     }
914 }
915

```

